

Diabetes Prediction Using Decision Tree Classification on the Kaggle Indian Diabetes Dataset

Table of Contents

1. Introduction & Objective

2. Indian Diabetes Dataset (Kaggle Dataset – Used for This Project)

3. Data Preprocessing & Splitting

4. Model Implementation & Training

5. Results & Evaluation

6. Discussion: Underfitting vs Overfitting

7. Deployment Demo (Gradio Application)

8. Conclusion

9. Appendix: Python Code

2.1 Exploratory Analysis

3.1 Data Preprocessing

3.2 Data Splitting

3.3 Summary

4.1 Model Setup

4.2 Training Process

4.3 Hyperparameter Tuning and Justification

5.1 Interpretation of Results

5.2 Comparison with Other Configurations

5.3 Conclusion on Model Performance

1. Introduction & Objective

Diabetes is a chronic metabolic disease that affects millions of people worldwide. Early detection of diabetes is crucial for preventing severe health complications. The primary objective of this project is to build a machine learning model capable of predicting whether a patient has diabetes (positive) or not (negative) based on various health metrics, using a real-world dataset. This is formulated as a **binary classification task**, where the target variable indicates the presence or absence of diabetes.

The chosen algorithm for this task is the **Decision Tree classifier**, which is interpretable, non-parametric, and effective for tabular medical datasets. While the original assignment suggested using the Scikit-learn diabetes dataset adapted for severity classification, we opted to use the **Indian Diabetes dataset from Kaggle**, which contains real patient health records. This provides a more realistic scenario for model training and evaluation, and allows us to explore the model's behavior on data closer to actual clinical settings. The model is trained to distinguish between diabetic and non-diabetic patients based on features such as glucose level, BMI, blood pressure, and age. Here's a polished version you can include in your report under **"2. Dataset Description & Exploration"**:

2 Indian Diabetes Dataset (Kaggle Dataset – Used for This Project)

To make the project more realistic and suitable for deployment, we used the **Indian Diabetes dataset from Kaggle**, which contains **768 patient records** and **8 clinical features** relevant to diabetes detection:

- Pregnancies
- Glucose
- Blood Pressure
- Skin Thickness
- Insulin
- BMI
- Diabetes Pedigree Function
- Age

The target variable is **binary**, indicating whether a patient is diagnosed with diabetes (1) or not (0).

Exploratory Analysis

A brief analysis of the Kaggle dataset shows:

- **Class distribution:** Approximately 65% non-diabetic and 35% diabetic patients, indicating a moderate class imbalance.
- **Missing or zero values:** Some features, such as glucose, blood pressure, insulin, and BMI, contain zero values, which are treated as missing data and replaced with the median of the respective feature.
- **Feature statistics:**
 - Glucose: mean ~121, range 0–199
 - BMI: mean ~32, range 0–67
 - Age: mean ~33, range 21–81

These statistics help understand the variability and scale of each feature, and guide preprocessing decisions for the Decision Tree classifier.

3. Data Preprocessing & Splitting

Before training the Decision Tree classifier, the Kaggle Indian Diabetes dataset required preprocessing to handle missing or inconsistent data and to prepare it for machine learning.

3.1 Data Preprocessing

- **Handling missing values:** In this dataset, certain features such as Glucose, Blood Pressure, Skin Thickness, Insulin, and BMI contain zero values, which are not physically meaningful and are treated as missing. These zeros were replaced with the **median value** of the corresponding feature. This approach is preferred for medical data to avoid skewing the distribution, while maintaining robustness against outliers.

- **Feature selection:** All 8 clinical features (Pregnancies, Glucose, Blood Pressure, Skin Thickness, Insulin, BMI, Diabetes Pedigree Function, and Age) were used as input features (X), while the target variable (y) indicates diabetes positive (1) or negative (0). No additional encoding was required since all features are numeric.
- **Data normalization:** Since Decision Trees are not sensitive to feature scaling, no normalization or standardization was applied.

3.2 Data Splitting

- The dataset was split into **training and testing sets** to evaluate the model's ability to generalize to unseen data.
- An **80/20 split** was used, meaning 80% of the data was used for training and 20% for testing.
- The `train_test_split` function from Scikit-learn was used with `random_state=42` to ensure reproducibility.

3.3 Summary

This preprocessing ensures that the model can learn meaningful patterns without being affected by missing values or inconsistent data. The training/testing split allows for **independent evaluation**, preventing data leakage and ensuring reliable performance metrics.

4. Model Implementation & Training

The model selected for this project is a **Decision Tree classifier**, which is a non-parametric, interpretable algorithm well-suited for tabular datasets and binary classification tasks. Decision Trees split the data based on feature thresholds to maximize class separation, making them ideal for understanding patterns in medical datasets like diabetes.

4.1 Model Setup

The Decision Tree was configured with the following **hyperparameters**, determined through experimentation and validation:

- `max_depth = 5` – limits the depth of the tree to prevent overfitting while allowing sufficient splits to capture patterns.
- `min_samples_split = 10` – requires at least 10 samples to split a node, reducing sensitivity to noise.
- `min_samples_leaf = 5` – ensures that each leaf node has at least 5 samples, preventing very small splits that could overfit.

These hyperparameters were chosen after testing multiple configurations and evaluating their effect on **training accuracy, test accuracy, precision, recall, and F1 score**.

4.2 Training Process

- The model was trained exclusively on the **training set (80% of the data)**.
- No feature scaling or encoding was needed, as all features are numeric and Decision Trees are invariant to scaling.
- Training involved recursively splitting nodes based on the **Gini impurity** criterion to maximize class purity in the leaves.

4.3 Hyperparameter Tuning and Justification

Several hyperparameter combinations were tested:

max_depth	min_samples_split	min_samples_leaf	Train Acc	Test Acc	F1 Score	Observation
1	50	20	0.76	0.69	0.39	Underfitting: too simple to learn patterns
2	1	1	1.00	0.67	0.46	Overfitting: memorizes training data, poor generalization
5	10	5	0.80	0.77	0.70	Optimal balance between bias and variance
5	5	5	0.80	0.77	0.70	Robust performance to small hyperparameter changes

Justification: The final configuration (`max_depth=5`, `min_samples_split=10`, `min_samples_leaf=5`) was selected because it **maximizes F1 score on the test set while maintaining good generalization**, avoids underfitting, and prevents overfitting. The tree depth is sufficient to capture relationships between features and diabetes outcomes, while the minimum samples constraints stabilize splits and improve robustness.

5. Results & Evaluation

After training the Decision Tree classifier with the selected hyperparameters (`max_depth=5`, `min_samples_split=10`, `min_samples_leaf=5`), the model was evaluated on the **test set (20% of the data)**. The key performance metrics are summarized below:

Metric	Value
Train Accuracy	0.801

Metric	Value
Precision	0.651
Recall	0.759
F1 Score	0.701

5.1 Interpretation of Results

- **Accuracy:** The test accuracy of 77.3% indicates that the model correctly classifies most patients as diabetic or non-diabetic.
- **Precision and Recall:** The model achieves higher recall (75.9%) than precision (65.1%), which is desirable in medical applications where identifying actual diabetic patients is critical.
- **F1 Score:** The balanced F1 score of 70.1% confirms that the model maintains a reasonable trade-off between precision and recall.

5.2 Comparison with Other Configurations

- **Underfitting:** Shallow trees with hyperparameters `max_depth=1`, `min_samples_split=50`, `min_samples_leaf=20` resulted in low recall (28%) and F1 (0.39), failing to detect many diabetic patients.
- **Overfitting:** Deep trees with `max_depth=15`, `min_samples_split=2`, `min_samples_leaf=1` achieved 100% training accuracy but poor test performance (~66.8% test accuracy, F1 ~0.46), memorizing training data without generalizing.
- **Optimal Model:** The selected configuration balances bias and variance, generalizes well, and ensures clinically relevant detection performance.

5.3 Conclusion on Model Performance

The Decision Tree classifier trained on the Kaggle Indian Diabetes dataset demonstrates **robust performance**, detecting the majority of diabetic patients while avoiding overfitting. The chosen hyperparameters provide a reliable and interpretable model suitable for deployment in a user-facing prediction application.

6. Discussion: Underfitting vs Overfitting

The experiments conducted with different Decision Tree configurations illustrate the **bias–variance tradeoff**:

- **Underfitting:** Shallow trees with high `min_samples_split` and `min_samples_leaf` values are too simple to capture patterns in the data. They exhibit **low training and test accuracy**, poor recall, and low F1 scores. This is an example of high bias, where the model cannot learn from the training data sufficiently.
- **Overfitting:** Deep trees with low `min_samples_split` and `min_samples_leaf` memorize the training data, achieving **perfect training accuracy**, but fail to generalize to unseen test data. This is an example of high variance, leading to poor test accuracy and unstable predictions.
- **Balanced Model:** The final model with `max_depth=5`, `min_samples_split=10`, `min_samples_leaf=5` strikes a balance between underfitting and overfitting. It generalizes well to the test set, maintains a high recall for diabetic patients, and provides a reliable F1 score.

Insight: This demonstrates the importance of **hyperparameter tuning** for Decision Trees, especially in medical datasets where misclassification of positive cases can have serious consequences.

7. Deployment Demo (Gradio Application)

To provide a real-world demonstration of the model, a **user-facing Gradio application** was developed:

- **Functionality:** Users can input individual patient metrics (e.g., glucose, BMI, age) into a simple interface.
- **Output:** The app predicts whether the patient is diabetic or non-diabetic.
- **Implementation:** The trained Decision Tree model is **loaded from a saved .pkl file**. No training or plots are exposed to the user, ensuring a clean, production-ready interface.
- **Benefit:** This demonstrates **practical deployment** of the model and makes the project suitable for portfolio presentation.

8. Conclusion

In this mini project, a **Decision Tree classifier** was successfully applied to the Kaggle Indian Diabetes dataset to predict whether a patient is diabetic. Key points:

- Preprocessing handled missing and zero values to ensure reliable inputs.
- Hyperparameter tuning explored the effects of tree depth and leaf constraints on underfitting and overfitting.
- The final model achieved **77.3% test accuracy** and a **balanced F1 score of 70.1%**, demonstrating robust generalization.
- The model was deployed via a **Gradio application**, allowing interactive user predictions in a real-world scenario.

This project demonstrates the complete **machine learning workflow (Split → Train → Predict → Evaluate)**, and highlights best practices for model selection, evaluation, and deployment in healthcare datasets.

9. Appendix: Python Code

Include all relevant Python scripts and notebook cells here. Suggested organization:

```
- 01_data_exploration.ipynb / .py
- 02_baseline_model.ipynb / .py
- train_model.py (training pipeline)
- diabetes_gradio_app.py (deployment)
```

- Show **data loading, preprocessing, model training, evaluation metrics calculation, hyperparameter tuning, and Gradio interface**.
- Include **tables and figures** as needed, such as the train/test accuracy table for different hyperparameters.